

# Reviewer Pack — Minimal Order of Etale Cohomology and DPLL Proof Length Ineq...

Entry b45ebb41c29b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 22:27:35 UTC

## Minimal Order of Etale Cohomology and DPLL Proof Length Inequality

**Entry ID:** b45ebb41c29b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 22:27:35 UTC

### 1. Conjecture

**Field A** (mathematical branch): Etale Cohomology **Field B** (complexity object): DPLL Proof Complexity

#### Statement:

For every CNF  $\varphi$  with  $n$  variables, the minimal order of etale cohomology group  $H^1(\mathbb{Q}_l, \varphi)$  is linearly correlated with its DPLL proof length  $l(\varphi)$ , such that  $\min_{x \in \mathbb{Z}_+} |H^1(\mathbb{Q}_l, \varphi)| = \Theta(l(\varphi))$ .

#### Rationale (proposer's reasoning):

Etale cohomology provides a powerful tool for studying algebraic varieties and their geometric properties. If there is a correlation between etale cohomology and the complexity of DPLL proof length, it may reveal new insights into the structure of CNFs and the hardness of SAT problems.

**Taxonomy category:** EtaleCohomologyToDPLLength (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 01336fb40ce19c85

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if at least 80% of the CNF instances have an absolute difference between their minimal order of etale cohomology and DPLL proof length less than or equal to 3, with a mean difference across all seeds not exceeding 3.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | HITS             | UNCERTAIN        |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | UNCERTAIN | 0.90       | UNCERTAIN        | HITS             |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "etale cohomology" AND "DPLL proof complexity" AND CNF" - "minimal order  $H^1(Q_l, \phi)$ " AND DPLL proof length" - "correlation etale cohomology DPLL proof"

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(10): # Generate 10 clauses with n variables
            clause = [random.randint(-n, -1), random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def dpll(cnf):
        def is_satisfiable(literals, assignment):
            for clause in cnf:
                if all(lit not in literals or (lit < 0 and -lit not in literals) for lit in clause):
                    continue
                else:
                    return False
            return True

        def backtrack(literals, assignment):
            if len(literals) == 0:
                return is_satisfiable(literals, assignment)

            literal = literals[0]
            if literal > 0:
                assignment[literal] = True
                if backtrack(literals[1:], assignment):
                    return True
                del assignment[literal]

            literal = literals[0]
            if literal < 0:
                assignment[literal] = False
                if backtrack(literals[1:], assignment):
                    return True
                del assignment[literal]

            return False

        return is_satisfiable([], assignment)

    return {'cnf': cnf, 'satisfiable': dpll(cnf)}
```

```

        assignment[-literal] = True
        if backtrack(literals[1:], assignment):
            return True
        del assignment[-literal]

    else:
        assignment[literal] = False
        if backtrack(literals[1:], assignment):
            return True
        del assignment[literal]

        assignment[-literal] = False
        if backtrack(literals[1:], assignment):
            return True
        del assignment[-literal]

literals = list(range(1, n + 1)) + [-i for i in range(1, n + 1)]
return backtrack(literals, {})

def etale_cohomology(cnf):
    # Simplified mapping to a graph problem (adjacency matrix)
    n = len(cnf)
    adj_matrix = [[0] * n for _ in range(n)]

    for clause in cnf:
        for i in range(len(clause)):
            for j in range(i + 1, len(clause)):
                u, v = abs(clause[i]) - 1, abs(clause[j]) - 1
                adj_matrix[u][v] = 1
                adj_matrix[v][u] = 1

    # Find the number of connected components (simplified)
    visited = [False] * n
    components = 0

    def dfs(u):
        stack = [u]
        while stack:
            u = stack.pop()
            if not visited[u]:
                visited[u] = True
                for v in range(n):
                    if adj_matrix[u][v] and not visited[v]:
                        stack.append(v)

    for i in range(n):
        if not visited[i]:
            dfs(i)
            components += 1

    return components

n_max = 40
instances_tested = 30
metric_values = []

```

```

for _ in range(instances_tested):
    cnf = generate_cnf(n_max)
    proof_length = dpll(cnf)
    cohomology_order = etale_cohomology(cnf)

    if proof_length is None or cohomology_order is None:
        return {
            "metric_name": "cohomology_diff",
            "metric_value": float('nan'),
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "dpll or cohomology failed"
        }

    metric_values.append(abs(cohomology_order - proof_length))

mean_diff = sum(metric_values) / len(metric_values)
support_fraction = sum(1 for diff in metric_values if abs(diff) <= 3) / len(metric_values)

return {
    "metric_name": "cohomology_diff",
    "metric_value": mean_diff,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_diff = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_diff} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"cohomology_diff\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=not_enough_data")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8037bd90.py", line 140, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8037bd90.py", line 106, in run_trial
    proof_length = dpll(cnf)
                  ^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8037bd90.py", line 64, in dpll
    literals = list(range(1, n + 1)) + [-i for i in range(1, n + 1)]
                  ^
NameError: name 'n' is not defined
```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the crash in the test code to proceed with the evaluation of the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 22462        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 9324         |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8413         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 9389         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14432        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11734        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 12513        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13541        |
| 9 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 12041        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 113848 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in `research/audit/b45ebb41c29b.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b45ebb41c29b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/b45ebb41c29b.tar.gz` (if generated)